

Model C Judicial Outcome Predictor

```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create a new session
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session

# Use the kagglehub client library to attach Kaggle resources like competitions, datasets, and models to your session
# Learn more about kagglehub: https://github.com/Kaggle/kagglehub/blob/main/README.md

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')
```

```
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_instruct_test.jsonl
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_v2_test.jsonl
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_b_as_c_test.jsonl
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_combined_test.jsonl
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_instruct_train.jsonl
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/03_segmented.csv
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_combined_train.jsonl
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_balanced_train.jsonl
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_fixed_train.jsonl
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_v2_val.jsonl
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_balanced_val.jsonl
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_b_as_c_train.jsonl
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_fixed_val.jsonl
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_b_as_c_val.jsonl
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_fixed_test.jsonl
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_balanced_test.jsonl
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_combined_val.jsonl
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_v2_train.jsonl
/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_instruct_val.jsonl
```

```
# =====
# CELL 1 – INSTALL + IMPORTS
# =====

!pip install -q scikit-learn joblib

import re
import json
import random
import joblib
import numpy as np

from collections import Counter

from sklearn.pipeline import Pipeline

from sklearn.svm import LinearSVC

from sklearn.feature_extraction.text import (
    TfidfVectorizer
)

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
)
```

```
# =====
# CELL 2 – LOAD DATASET
# =====
```

```

def load_jsonl(path):

    records = []

    with open(
        path,
        encoding="utf-8"
    ) as f:

        for line in f:

            line = line.strip()

            if line:

                records.append(
                    json.loads(line)
                )

    return records

train_data = load_jsonl(
    "/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_combined_train.jsonl"
)

val_data = load_jsonl(
    "/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_combined_val.jsonl"
)

test_data = load_jsonl(
    "/kaggle/input/datasets/mayaralo/datasetmodelcfiles/model_c_combined_test.jsonl"
)

print("=" * 60)
print("DATA LOADED")
print("=" * 60)

print("Train:", len(train_data))
print("Validation:", len(val_data))
print("Test:", len(test_data))

```

```

=====
DATA LOADED
=====
Train: 1550
Validation: 375
Test: 702

```

```

# =====
# CELL 3 – DATA DISTRIBUTION
# =====

all_labels = [

    r["verdict_label"]

    for r in (
        train_data
        + val_data
        + test_data
    )
]

counter = Counter(all_labels)

print("=" * 60)
print("LABEL DISTRIBUTION")
print("=" * 60)

for k, v in counter.items():

    print(f"{k}: {v}")

```

```

=====
LABEL DISTRIBUTION
=====
accepted: 1636
rejected: 872
partial: 119

```

```

# =====
# CELL 4 – LEGAL STOPWORDS

```

```
# =====

arabic_stopwords = [

    "المدعي",
    "المدعى",
    "المدعية",
    "المدعى عليه",
    "المدعى عليها",
    "الدعوى",
    "القضية",
    "الدائرة",
    "المحكمة",
    "عليه",
    "عليها",
    "من",
    "الى",
    "على",
    "وقد",
    "انه",
    "في",
    "هذا",
    "هذه",
    "ذلك",
    "يما",
    "ان",
    "با",
    "تم",
    "رقم",
    "قد",
    "ريالا",
    "مبلغ",
    "وقدره",

]
```

```
# =====
# IMPROVED CLEAN TEXT
# =====

def clean_text(text):

    if text is None:
        return ""

    text = str(text)

    # lowercase
    text = text.lower()

    # إزالة التشكيل
    text = re.sub(
        r'[\u0617-\u061A\u064B-\u0652]',
        '',
        text
    )

    # إزالة الرموز
    text = re.sub(
        r'^\w\s]',
        '',
        text
    )

    # إزالة الأرقام
    text = re.sub(
        r'\d+',
        '',
        text
    )

    # =====
    # LEGAL KEYWORD BOOSTING
    # =====

    legal_terms = {

        # =====
        # STRONG ACCEPTANCE
        # =====

        "عقد قوي": "عقد قوي",
```

```

    "عقد_قوي " : "اتفاقية",
    "اثبات_مالي " : "فاتورة",
    "اثبات_مالي " : "فواتير",
    "استلام_رسمي " : "محضر استلام",
    "استلام_رسمي " : "محاضر الاستلام",
    "اقرار " : "اقر",
    "اقرار " : "أقر",
    "اقرار " : "اقرت",
    "اقرار " : "أقرت",

# =====
# NEGATIVE LEGAL SIGNALS
# =====

    "غياب_اثبات " : "لم يقدم",
    "غياب_اثبات " : "عدم تقديم",
    "غياب_اثبات " : "لا يوجد",
    "غياب_اثبات " : "عدم وجود",
    "غياب_اثبات " : "عدم ثبوت",
    "غياب_اثبات " : "عدم كفاية",
    "نفي_قوي " : "ينكر",
    "دفع_قوي " : "قوة قاهرة",
    "رفض_صریح " : "رفض",
}

for k, v in legal_terms.items():

    text = text.replace(
        k,
        v
    )

# =====
# REMOVE LEGAL FILLERS
# =====

remove_phrases = [

    "بعد الاطلاع",
    "وحيث إن",
    "لدى الدائرة",
    "فإن الدائرة",
    "ويعد التأمل",
    "وبعد دراسة أوراق الدعوى",
    "بعد دراسة القضية",
    "وبناء عليه",
    "لذلك حكمت الدائرة",
]

for phrase in remove_phrases:

    text = text.replace(
        phrase,
        ""
    )

# تنظيف الفراغات
text = re.sub(
    r'\s+',
    ' ',
    text
)

return text.strip()

```

```

# =====
# CELL 6 – PREPARE DATA

```

```
# =====

all_train = (
    train_data
    + val_data
)

X_train = [
    clean_text(
        r["input"]
    )

    for r in all_train
]

y_train = [
    r["verdict_label"]

    for r in all_train
]

print("=" * 60)
print("TRAIN DATA READY")
print("=" * 60)

print("Samples:", len(X_train))
```

```
=====
TRAIN DATA READY
=====
Samples: 1925
```

```
# =====
# CELL 7 – BUILD MODEL
# =====

model = Pipeline([
    (
        "tfidf",
        TfidfVectorizer(
            max_features=30000,
            ngram_range=(1,3),
            min_df=2,
            max_df=0.95,
            sublinear_tf=True,
            stop_words=arabic_stopwords,
        )
    ),
    (
        "clf",
        LinearSVC(
            class_weight={
                "accepted": 1,
                "rejected": 4,
            },
            max_iter=5000,
        )
    )
])

print("Model ready!")
```

```
Model ready!
```

```
# =====
# CELL 8 – TRAIN MODEL
# =====

print("=" * 60)
print("TRAINING MODEL...")
print("=" * 60)

model.fit(
    X_train,
    y_train
)

print("Training complete!")
```

```
=====
TRAINING MODEL...
=====
Training complete!
```

```
# =====
# CELL 9 – BALANCED TEST SET
# =====

accepted_cases = [
    r for r in test_data
    if r["verdict_label"] == "accepted"
]

rejected_cases = [
    r for r in test_data
    if r["verdict_label"] == "rejected"
]

random.seed(42)

sample = (
    random.sample(
        accepted_cases,
        min(100, len(accepted_cases))
    )
    +
    random.sample(
        rejected_cases,
        min(40, len(rejected_cases))
    )
)

random.shuffle(sample)

X_test = [
    clean_text(
        r["input"]
    )
    for r in sample
]

y_test = [
    r["verdict_label"]
    for r in sample
]

print("Balanced test set ready!")
```

```
Balanced test set ready!
```

```
# =====
# CELL 10 – EVALUATION
# =====

y_pred = model.predict(
```

```

    X_test
)

acc = accuracy_score(
    y_test,
    y_pred
)

print("=" * 60)
print("FINAL RESULTS")
print("=" * 60)

print(
    f"Accuracy: {acc:.2%}"
)

print()

print(
    classification_report(
        y_test,
        y_pred,
        digits=4,
        zero_division=0
    )
)

```

```

=====
FINAL RESULTS
=====
Accuracy: 72.86%

```

	precision	recall	f1-score	support
accepted	0.7500	0.9300	0.8304	100
rejected	0.5625	0.2250	0.3214	40
accuracy			0.7286	140
macro avg	0.6562	0.5775	0.5759	140
weighted avg	0.6964	0.7286	0.6849	140

```

# =====
# EXPLAIN CASE FUNCTION
# FINAL STABLE LEGAL VERSION
# =====

def explain_case(case_text):

    cleaned = clean_text(case_text)

    prediction = model.predict(
        [cleaned]
    )[0]

    # -----
    # ACCEPTED
    # -----

    strong_accept = (
        (
            "عقد قوي" in cleaned
            or
            "استلام رسمي" in cleaned
        )
        and
        "اثبات مالي" in cleaned
    )

    # -----
    # REJECTED
    # -----

```

```

strong_reject = (
    "غياب_اثبات" in cleaned
    and
    "عقد_قوي" not in cleaned
    and
    "استلام_رسمي" not in cleaned
)

# -----
# PARTIAL
# -----

strong_partial = (
    (
        "المبلغ المتبقي" in cleaned
        or
        "المتبقي" in cleaned
        or
        "جزء" in cleaned
        or
        "اعترض" in cleaned
        or
        "تدقيق الحسابات" in cleaned
        or
        "سدد" in cleaned
    )
    and
    "عقد_قوي" in cleaned
)

# =====
# APPLY RULES
# =====

# partial أولاً
if strong_partial:
    prediction = "partial"

# قبول قوي
elif strong_accept:
    prediction = "accepted"

# رفض قوي
elif strong_reject:
    prediction = "rejected"

# =====
# MODEL OBJECTS
# =====

vectorizer = model.named_steps[
    "tfidf"
]

classifier = model.named_steps[
    "clf"
]

X = vectorizer.transform(
    [cleaned]
)

```



```

)

# =====
# CONFIDENCE SCORE
# =====

decision = classifier.decision_function(X)

if np.ndim(decision) == 1:

    confidence = abs(
        decision[0]
    )

else:

    confidence = np.max(
        np.abs(decision)
    )

confidence = float(confidence)

confidence = min(
    confidence * 20,
    100
)

confidence = round(
    confidence,
    2
)

# =====
# UNCERTAIN CASES
# =====

if confidence < 25 and prediction not in [
    "accepted",
    "rejected",
    "partial"
]:

    prediction = "uncertain"

# =====
# IMPORTANT FEATURES
# =====

feature_names = (
    vectorizer
    .get_feature_names_out()
)

weights = X.toarray()[0]

important = []

for idx, value in enumerate(weights):

    if value > 0:

        coef = classifier.coef_[0][idx]

        score = value * coef

        important.append(
            (
                feature_names[idx],
                score
            )
        )

important = sorted(

    important,

    key=lambda x: abs(x[1]),

    reverse=True
)

```

```

top_words = [
    w[0]
    for w in important[:10]
]

# =====
# DISPLAY RESULTS
# =====

print()
print("=" * 60)
print("LEGAL CASE ANALYSIS")
print("=" * 60)

print(
    "Prediction:",
    prediction
)

print(
    "Confidence:",
    f"{confidence}%"
)

print()

# =====
# EXPLANATION
# =====

if prediction == "accepted":

    print("Explanation:")

    print(
        "تم تصنيف القضية كمقبولة"
        " لوجود أدلة قوية مثل العقود"
        " أو الفواتير أو محاضر الاستلام"
    )

elif prediction == "rejected":

    print("Explanation:")

    print(
        "تم تصنيف القضية كمرفوضة"
        " بسبب ضعف الإثبات أو"
        " عدم كفاية الأدلة المقدمة"
    )

elif prediction == "partial":

    print("Explanation:")

    print(
        "تم تصنيف القضية كمقبولة جزئياً"
        " لثبوت أصل الحق"
        " مع وجود نزاع على جزء من المطالبة"
    )

else:

    print("Explanation:")

    print(
        "القضية تحتوي على مؤشرات"
        " متعارضة ولا يمكن الجزم"
        " بالحكم بثقة عالية"
    )

# =====
# IMPORTANT FEATURES
# =====

print()
print("=" * 60)
print("IMPORTANT FEATURES")
print("=" * 60)

for w in top_words:

    print("-", w)

```

LEGAL CASE ANALYSIS

Prediction: accepted
Confidence: 13.49%

Explanation:

تم تصنيف القضية كمقبولة لوجود أدلة قوية مثل العقود أو الفواتير أو محاضر الاستلام

IMPORTANT FEATURES

- توريد
- للمقاولات
- مواد بناء
- المستندات
- عقود قوي
- صحة
- عن
- اثبات مالي
- الطلبات
- مواد

```
# =====
# EXPLAIN CASE FUNCTION
# FINAL VERSION WITH SMART LEGAL REASONING
# =====

def explain_case(case_text):

    # =====
    # CLEAN TEXT
    # =====

    cleaned = clean_text(case_text)

    # =====
    # ML PREDICTION
    # =====

    prediction = model.predict(
        [cleaned]
    )[0]

    # =====
    # LEGAL REASONING RULES
    # =====

    # -----
    # ACCEPTED
    # -----

    strong_accept = (

        (
            "عقد قوي" in cleaned
            or
            "استلام رسمي" in cleaned
        )

        and

        "اثبات مالي" in cleaned
    )

    # -----
    # REJECTED
    # -----

    strong_reject = (

        "غياب اثبات" in cleaned

        and

        "عقد قوي" not in cleaned

        and
```

```

    "استلام_رسمي" not in cleaned
)

# -----
# PARTIAL
# -----

strong_partial = (
    (
        "المبلغ المتبقي" in cleaned
        or
        "المتبقي" in cleaned
        or
        "جزء" in cleaned
        or
        "اعتراض" in cleaned
        or
        "تدقيق الحسابات" in cleaned
        or
        "سدد" in cleaned
    )
    and
    "عقد_قوي" in cleaned
)

# =====
# APPLY RULES
# =====

if strong_partial:
    prediction = "partial"
elif strong_accept:
    prediction = "accepted"
elif strong_reject:
    prediction = "rejected"

# =====
# MODEL OBJECTS
# =====

vectorizer = model.named_steps[
    "tfidf"
]

classifier = model.named_steps[
    "clf"
]

X = vectorizer.transform(
    [cleaned]
)

# =====
# CONFIDENCE SCORE
# =====

decision = classifier.decision_function(X)

if np.ndim(decision) == 1:
    confidence = abs(
        decision[0]
    )

```

```

else:

    confidence = np.max(
        np.abs(decision)
    )

confidence = float(confidence)

confidence = min(
    confidence * 20,
    100
)

confidence = round(
    confidence,
    2
)

# =====
# UNCERTAIN CASES
# =====

if confidence < 25 and prediction not in [
    "accepted",
    "rejected",
    "partial"
]:

    prediction = "uncertain"

# =====
# IMPORTANT FEATURES
# =====

feature_names = (
    vectorizer
    .get_feature_names_out()
)

weights = X.toarray()[0]

important = []

for idx, value in enumerate(weights):

    if value > 0:

        coef = classifier.coef_[0][idx]

        score = value * coef

        important.append(
            (
                feature_names[idx],
                score
            )
        )

important = sorted(

    important,

    key=lambda x: abs(x[1]),

    reverse=True
)

top_words = [
    w[0]
    for w in important[:10]
]

# =====
# SMART LEGAL REASONING
# =====

reasoning = generate_legal_reasoning(
    prediction,
    cleaned
)

```

```

# =====
# DISPLAY RESULTS
# =====

print()
print("=" * 60)
print("LEGAL CASE ANALYSIS")
print("=" * 60)

print(
    "Prediction:",
    prediction
)

print(
    "Confidence:",
    f"{{confidence}}%"
)

print()

print("Explanation:")

print(reasoning)

# =====
# IMPORTANT FEATURES
# =====

print()
print("=" * 60)
print("IMPORTANT FEATURES")
print("=" * 60)

for w in top_words:

    print("-", w)

```

```
facts = """
```

الوقائع:
تدور القضية حول نزاع توريد بين المدعي شركة النخبة للمقاولات
والمدعى عليه **.

يطالب المدعي بسداد مبلغ 450,000 ٠
مستحق عن عقد توريد مواد بناء منفذ بالكامل.

وقدّم المدعي:
عقد التوريد الموقع -
فواتير التسليم -
محاضر الاستلام -

ينازع المدعى عليه في صحة المستندات
ويدفع بعدم ثبوت المديونية.

الطلبات:
إلزام المدعى عليه بسداد مبلغ 450,000 ٠

الدفع:
ينكر المدعى عليه وجود عقد ملزم
ويطعن في صحة المستندات.

```
"""
```

```
explain_case(facts)
```

```
=====
LEGAL CASE ANALYSIS
=====
```

```
Prediction: accepted
Confidence: 13.49%
```

```
Explanation:
```

تم قبول الدعوى بسبب ثبوت العلاقة التعاقدية و وجود مستندات مالية مؤيدة و ثبوت استلام الأعمال أو البضاعة

```
=====
IMPORTANT FEATURES
=====
```

```
- توريد
- للمقاولات
- مواد بناء
```

- المستندات
- عقد قوي
- صحة
- عن
- اثبات مالي
- الطلبات
- مواد

```
# =====
# TEST CASE 2
# =====

facts = ""

الوقائع:
تدور القضية حول مطالبة المدعي
بتعويض عن أضرار ناجمة
عن تأخر تسليم مشروع إنشائي.

يطالب المدعي بمبلغ
300,000 ١
.تعويضاً عن الخسائر.

لم يقدم المدعي أي مستندات
تثبت الضرر الفعلي
.أو تقارير تقييم الأضرار.

الطلبات:
إلزام المدعى عليه
.بدفع 300,000 ١ ٢
تعويضاً.

الدفع:
يدفع المدعى عليه
بأن التأخير كان بسبب قوة القاهرة
.خارجة عن إرادته.

كما يدفع بأن المدعي
لم يقدم أي دليل
.يثبت وقوع الضرر الفعلي.

""

explain_case(facts)
```

```
=====
LEGAL CASE ANALYSIS
=====
Prediction: rejected
Confidence: 17.41%

Explanation:
.تم رفض الدعوى بسبب عدم كفاية الأدلة و وجود سبب نظامي أو قوة القاهرة

=====
IMPORTANT FEATURES
=====
- عن
- بمبلغ
- الضرر
- مشروع
- الطلبات
- يدفع
- غياب اثبات
- تثبت
- بسبب
- الخسائر
```

facts = ""

الوقائع:

• تدور القضية حول نزاع شراكة بين المدعي والمدعى عليه في مشروع تجاري مشترك.

• يطالب المدعي بحصته من الأرباح البالغة 150,000 ١.

• أقر المدعى عليه بوجود الشراكة لكنه ينازع في مقدار الأرباح المستحقة.

• قدم المدعي عقد الشراكة فقط دون كشوف حسابية أو تقارير مالية.

الطلبات:

• إلزام المدعى عليه بسداد 150,000 ١ حصة في الأرباح.

الدفع:

• يقر المدعى عليه بالشراكة لكنه يؤكد أن الأرباح الفعلية أقل مما يدعيه المدعي.

• ويطالب بتدقيق الحسابات.

""

explain_case(facts)

=====

LEGAL CASE ANALYSIS

=====

Prediction: partial

Confidence: 17.0%

Explanation:

• تم قبول الدعوى جزئياً بسبب ثبوت أصل العلاقة القانونية و الحاجة إلى مراجعة الحسابات.

=====

IMPORTANT FEATURES

=====

- اقرار
- دون
- الشراكة
- فقط
- المستحقة
- مشروع
- الطلبات
- مقدار
- عقدي
- قوي
- البالغة

facts = ""

الوقائع:

• تدور القضية حول مطالبة المدعي شركة البناء الحديث بمبلغ 800,000 ١.

• قيمة أعمال مقاوله منجزة بموجب عقد مبرم بين الطرفين.

• قدم المدعي عقد المقاولة وشهادات إنجاز الأعمال وفواتير المستخلصات.

• سدد المدعى عليه مبلغ 500,000 ١ واعترض على المبلغ المتبقي.

الطلبات:

• إلزام المدعى عليه بسداد المبلغ المتبقي 300,000 ١.

الدفع:

• يقر المدعى عليه بالعقد وبسداد 500,000 ١،

• لكنه يدفع بأن الأعمال المتبقية لم تكتمل وفق المواصفات المتفق عليها.

""

explain_case(facts)

=====

LEGAL CASE ANALYSIS

=====

Prediction: partial

Confidence: 18.14%

Explanation:

• قبول الدعوى جزئياً بسبب ثبوت أصل العلاقة القانونية و وجود اعتراض على جزء من المطالبة و وجود نزاع حول المبلغ المتبقي.

=====

IMPORTANT FEATURES

=====

- المبلغ
- المبلغ المتبقي
- بمبلغ
- عقدي
- اثبات مالي
- لم
- الطلبات
- بسداد المبلغ
- قيمة
- المتبقية


```

# =====
# SMART LEGAL EXPLANATION GENERATOR
# =====

def generate_legal_reasoning(prediction, cleaned_text):

    # =====
    # ACCEPTED
    # =====

    if prediction == "accepted":

        reasons = []

        if "عقد_قوي" in cleaned_text:

            reasons.append(
                "ثبوت العلاقة التعاقدية"
            )

        if "اثبات_مالي" in cleaned_text:

            reasons.append(
                "وجود مستندات مالية مؤيدة"
            )

        if "استلام_رسمي" in cleaned_text:

            reasons.append(
                "ثبوت استلام الأعمال أو البضاعة"
            )

        if "اقرار" in cleaned_text:

            reasons.append(
                "وجود إقرار من المدعى عليه"
            )

        if len(reasons) == 0:

            reasons.append(
                "وجود مؤشرات قانونية داعمة للمطالبة"
            )

        return (
            "تم قبول الدعوى بسبب "
            + " ".join(reasons)
            + " ."
        )

    # =====
    # REJECTED
    # =====

    elif prediction == "rejected":

        reasons = []

        if "غياب_اثبات" in cleaned_text:

            reasons.append(
                "عدم كفاية الأدلة"
            )

        if "نفي_قوي" in cleaned_text:

            reasons.append(
                "وجود دفع مؤثرة من المدعى عليه"
            )

        if "دفع_قوي" in cleaned_text:

            reasons.append(
                "وجود سبب نظامي أو قوة قاهرة"
            )

        if len(reasons) == 0:

            reasons.append(
                "ضعف الإثبات القانوني"
            )

```

```

        return (
            "تم رفض الدعوى بسبب"
            + " ".join(reasons)
            + "."
        )

# =====
# PARTIAL
# =====

elif prediction == "partial":

    reasons = []

    reasons.append(
        "ثبوت أصل العلاقة القانونية"
    )

    if "اعتراض" in cleaned_text:

        reasons.append(
            "وجود اعتراض على جزء من المطالبة"
        )

    if "المبلغ المتبقي" in cleaned_text:

        reasons.append(
            "وجود نزاع حول المبلغ المتبقي"
        )

    if "تدقيق الحسابات" in cleaned_text:

        reasons.append(
            "الحاجة إلى مراجعة الحسابات"
        )

    return (
        "تم قبول الدعوى جزئياً بسبب"
        + " ".join(reasons)
        + "."
    )

# =====
# UNCERTAIN
# =====

else:

    return (
        "تعذر الوصول إلى نتيجة قانونية"
        + "حاسبة بسبب تعارض المؤشرات"
    )

```

```

# =====
# SAVE FINAL MODEL
# =====

import joblib

joblib.dump(
    model,
    "final_legal_ai_model2.pkl"
)

print("FINAL LEGAL AI MODEL SAVED")

print("Saved as:")
print("final_legal_ai_model.pkl")

```

FINAL LEGAL AI MODEL SAVED
Saved as: